

Recursive bisection

Recursive bisection builds a multiway partition by repeatedly bipartitioning the largest part until you reach k parts

The idea

- Each step is an ordinary bipartition on an induced subgraph
- Quality compounds: a weak first cut leaves later bisectors fewer good options
- Always report total cut across all part boundaries and the part-size vector

Pseudocode

- Recursive bisection is a loop over parts
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- First split yields ABC versus DE at cut three
- For k equals three the sketch bisects ABC next into A B versus C with D E untouched

Algorithm sketch — try these

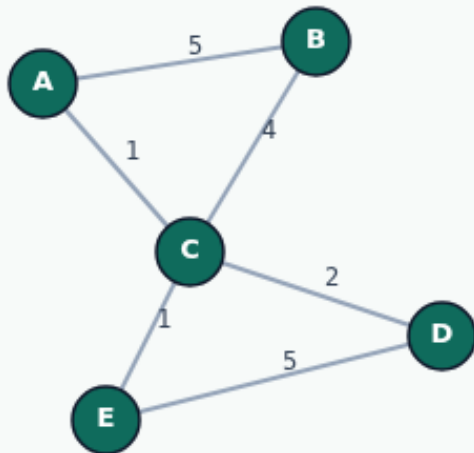
```
INPUT: G, target k parts
OUTPUT: side[v] ∈ {0..k-1}
parts ← {all nodes}
while |parts| < k:
    pick largest part P
    bipartition P (spectral/KL/FM)
    replace P with the two halves
GOLDEN k=2: ABC|DE cut=3
k=3 continues on ABC → AB|C|DE
```

Start: one part, k=4 target

RECURSIVE BISECTION

Start: one part, k=4 target

Step 1 / 5 · start · module02-07-recursive-bisection



Explanation

Recursive bisection builds k-way partitions by repeated 2-way splits. Target k=4 on TINY_GRAPH: bisect the largest part until four labels exist.

- Always split the largest current part
- Inner split uses spectral bisection
- History logs each bisection step

```
target k=4
step 0: ABCDE cut=0
```

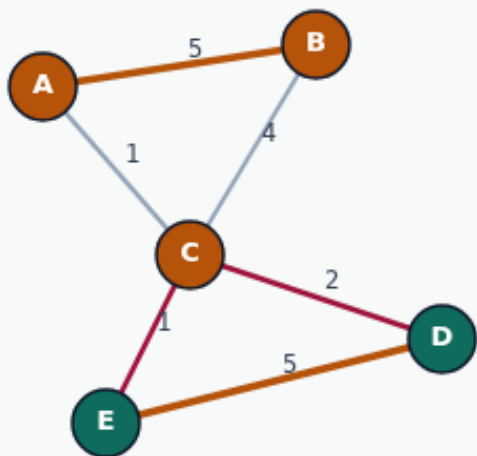
Start: one part, k=4 target

Bisect 1: ABC|DE cut 3

RECURSIVE BISECTION

Bisect 1: ABC|DE cut 3

Step 2 / 5 · split-1 · module02-07-recursive-bisection



Explanation

First spectral split on the whole graph yields ABC|DE — same golden bipartition with cutsize 3. Two parts remain; need two more splits.

- Largest part = all five nodes
- Spectral → ABC vs DE
- Heavy edges A-B, D-E internal

step 1: ABC|DE
cutsize: 3
k=2 so far

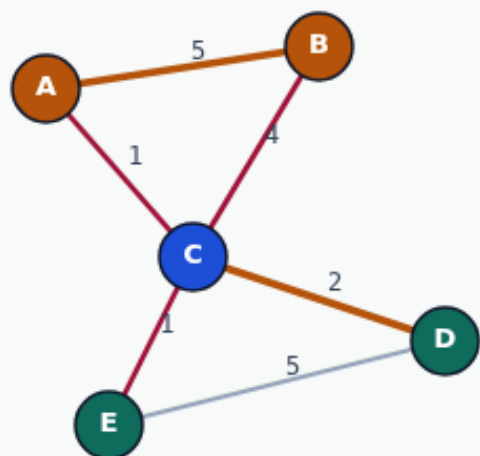
Bisect 1: ABC|DE cut 3

Bisect 2: split ABC → AB|C

RECURSIVE BISECTION

Bisect 2: split ABC → AB|C

Step 3 / 5 · split-2 · module02-07-recursive-bisection



Explanation

Largest part ABC (size 3) is bisected next. A and B stay together ($w=5$); C becomes its own part. Three parts: AB, C, DE.

- Split ABC via induced spectral
- A-B heavy edge survives
- cutsize rises to 8

step 2: AB|C|DE
cutsize: 8
k=3

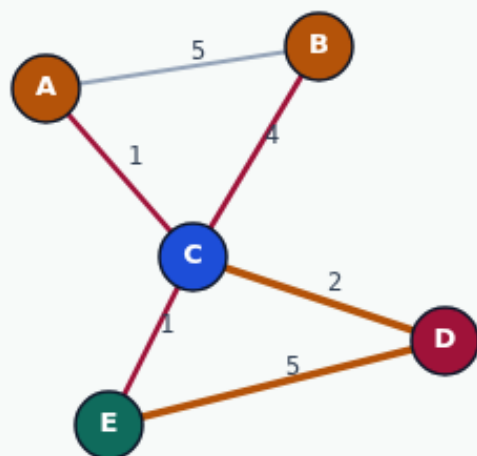
Bisect 2: split ABC → AB|C

Bisect 3: split DE → D|E

RECURSIVE BISECTION

Bisect 3: split DE → D|E

Step 4 / 5 · split-3 · module02-07-recursive-bisection



Explanation

Largest remaining part DE is bisected, cutting the weight-5 D-E edge. Final four parts: AB, C, D, E singleton aside from AB pair.

- Split DE — pays the heavy edge
- $k=4$ reached
- AB still uncut

step 3: AB|C|D|E
cutsizes: 13
 $k=4$

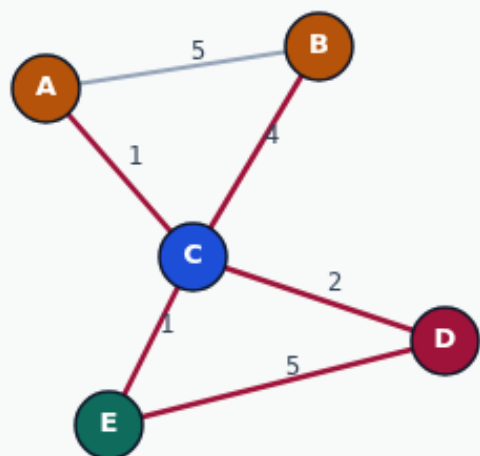
Bisect 3: split DE → D|E

Recursive bisection tradeoff

RECURSIVE BISECTION

Recursive bisection tradeoff

Step 5 / 5 · takeaway · module02-07-recursive-bisection



Explanation

k=4 costs cut 13 because splitting DE destroys the weight-5 edge. k=3 stops earlier at AB|C|DE cut 8 — fewer splits can mean lower total cut.

- Largest-part-first scheduling
- Each split is local spectral
- Compare k=3 (cut 8) vs k=4 (cut 13)

Final: AB|C|D|E cut=13
k=3 reference: cut=8

Browser lab track

- In the browser lab track, open the **recursive-bisection** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module